

Solution Architecture

Project Title

Virtual Agent–Driven SLA Breach Awareness & Justification System

Introduction

Solution Architecture defines the overall structure of the Virtual Agent–Driven SLA Breach Awareness & Justification System. It describes how different software components interact to provide proactive SLA monitoring, workflow automation, AI-assisted decision support, and real-time analytics. The architecture follows a layered approach to ensure scalability, maintainability, security, and efficient communication between the user interface, application logic, AI services, and database.

Architecture Overview

The application follows a **three-tier architecture** consisting of:

1. Presentation Layer
2. Application Layer
3. Data Layer

Each layer performs specific responsibilities while communicating with the other layers to deliver a seamless user experience.

Architecture Components

1. Presentation Layer

The Presentation Layer is the interface through which users interact with the application.

Technologies Used

- HTML5
- CSS3
- JavaScript
- Chart.js
- ServiceNow Service Portal
- Virtual Agent Interface

Responsibilities

- User authentication
- Incident dashboard
- SLA status display

- Ticket management
 - Virtual Agent interaction
 - Report visualization
 - User notifications
-

2. Application Layer

The Application Layer contains the core business logic of the system.

Technologies Used

- Python
- Flask Framework
- Google Gemini AI
- ServiceNow Flow Designer

Responsibilities

- Process user requests
 - Manage incident workflows
 - Monitor SLA timers
 - Classify SLA status
 - Generate AI-based justifications
 - Execute automated workflows
 - Trigger notifications
 - Generate dashboard data
-

3. Data Layer

The Data Layer stores and manages application data.

Technology Used

SQLite Database

Data Stored

- User information
- Incident records
- SLA details
- Ticket history

- Notification logs
 - AI-generated justifications
 - Dashboard statistics
 - Audit records
-

System Workflow

The overall workflow of the application is as follows:

Step 1

User logs into the application.



Step 2

Incident is created or updated.



Step 3

The system automatically associates the appropriate SLA.



Step 4

The SLA Monitoring Engine continuously tracks the remaining SLA time.



Step 5

The system classifies the incident as:

- SAFE
- AT RISK
- BREACHED



Step 6

Flow Designer triggers automated notifications when SLA thresholds are reached.



Step 7

The Virtual Agent informs the assigned support agent about the SLA status.



Step 8

Google Gemini AI generates an intelligent SLA breach explanation and recommends corrective actions.



Step 9

The support agent acknowledges the SLA risk and updates the incident with the required justification.



Step 10

The system stores all updates and refreshes dashboards and reports.

Component Interaction

User

The user accesses the application through a web browser or the ServiceNow Service Portal.



Frontend

The frontend collects user input and displays incident details, dashboards, notifications, and Virtual Agent conversations.



Flask Application

Flask processes requests, executes business logic, communicates with the database, and integrates with AI services.



Google Gemini AI

Gemini AI analyzes incident context and generates SLA breach explanations and recommended actions.



SQLite Database

Stores all application data, including incidents, SLA information, users, notifications, and AI-generated justifications.



Dashboard

Displays updated SLA metrics, incident statistics, and performance reports.

Security Architecture

The application implements multiple security mechanisms to protect organizational data.

Authentication

- Secure user login
- Session management
- Credential verification

Authorization

Role-Based Access Control (RBAC) ensures controlled access.

Roles

- Administrator
- Team Lead / Manager
- Support Agent

Data Security

- Secure database storage
- Controlled user permissions
- Audit trail maintenance
- Incident history tracking

Notification Architecture

The notification module automatically alerts users during critical SLA events.

Notification Triggers

- SLA reaches warning threshold
- SLA is breached
- Incident assigned
- Incident updated
- Justification required
- Workflow completed

Notification Channels

- Virtual Agent
- Service Portal

- Email Notifications
-

AI Integration Architecture

Google Gemini AI integrates with the application to provide intelligent decision support.

Input

- Incident details
- SLA status
- Remaining SLA time
- Incident priority
- Current workflow stage

Processing

- Analyze incident context
- Generate SLA breach explanation
- Recommend corrective actions
- Produce user-friendly responses

Output

- AI-generated justification
 - Corrective guidance
 - Virtual Agent response
-

Advantages of the Architecture

The proposed architecture offers several advantages:

- Modular design
- Easy maintenance
- High scalability
- Secure user management
- Automated workflow execution
- AI-assisted decision support
- Real-time SLA monitoring
- Efficient data management
- Interactive dashboards

- Improved operational efficiency
-

Future Architectural Enhancements

The architecture can be extended to support:

- Cloud database integration
 - Machine Learning prediction models
 - Mobile application support
 - REST API integration with enterprise ITSM tools
 - Multi-channel notification services
 - Advanced Business Intelligence dashboards
 - Distributed deployment for large organizations
-

Conclusion

The Solution Architecture provides a structured framework for implementing the **Virtual Agent–Driven SLA Breach Awareness & Justification System**. By integrating modern web technologies, ServiceNow workflows, Google Gemini AI, and a secure database, the architecture enables proactive SLA management, intelligent decision support, automated notifications, and real-time analytics. The modular design ensures that the application remains scalable, maintainable, and adaptable to future organizational requirements.